

PROGRAM 4

TRANSFER LEARNING USING VGG16

OBJECTIVE

To perform Image Classification using a pre-trained VGG16 model and custom classification layers.

In this program students will learn:

- ✓ Reading Dataset from Folders
 - ✓ Transfer Learning
 - ✓ Loading .weights.h5 Files
 - ✓ Feature Extraction
 - ✓ Freezing Layers
 - ✓ Custom Classification Layers
 - ✓ Model Training
 - ✓ Model Evaluation
 - ✓ Prediction
-

MODEL FILE REQUIRED

Models/

└── vgg16.weights.h5

DATASET FOLDER STRUCTURE

Fruits/

```
├── Apple/
|   ├── image1.jpg
|   ├── image2.jpg
|   └── ...
|
├── Banana/
|   ├── image1.jpg
|   ├── image2.jpg
|   └── ...
|
└── Orange/
```

```
├── image1.jpg
├── image2.jpg
└── ...
```

Explanation:

Each folder represents one class.

The folder name automatically becomes the class label.

STEP 1 IMPORT LIBRARIES

Purpose:

Import required packages.

Code:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout

print("Libraries Imported Successfully")
```

STEP 2 LOAD DATASET

Purpose:

Read images from folders.

Code:

```
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_data = datagen.flow_from_directory(
    "Fruits",
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    subset='training'
)

val_data = datagen.flow_from_directory(
    "Fruits",
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    subset='validation'
)
```

```
print("Dataset Loaded")
```

Explanation:

VGG16 requires:

224 × 224 images

Therefore all images are resized accordingly.

STEP 3 DISPLAY SAMPLE IMAGES

Purpose:

Verify dataset loading.

Code:

```
images, labels = next(train_data)

plt.figure(figsize=(10,5))

for i in range(6):

    plt.subplot(2,3,i+1)

    plt.imshow(images[i])

    plt.axis("off")

plt.show()
```

STEP 4 CHECK CLASS LABELS

Code:

```
print(  
    train_data.class_indices  
)
```

Expected Output:

```
{  
  'Apple':0,  
  'Banana':1,  
  'Orange':2  
}
```

STEP 5 LOAD VGG16 ARCHITECTURE

Purpose:

Create VGG16 architecture.

Code:

```
base_model = VGG16(  
    weights=None,  
    include_top=False,  
    input_shape=(224,224,3)  
)  
  
print("VGG16 Architecture Loaded")
```

Explanation:

weights=None

Prevents internet download.

include_top=False

Removes original ImageNet classifier.

STEP 6 LOAD .WEIGHTS.H5 FILE

Purpose:

Load pretrained weights.

Code:

```
base_model.load_weights(  
    "Models/vgg16.weights.h5"  
)  
  
print("Weights Loaded Successfully")
```

Explanation:

The model learns from previously trained ImageNet weights.

This is the main idea of Transfer Learning.

STEP 7 FREEZE PRETRAINED LAYERS

Purpose:

Prevent modification of pretrained weights.

Code:

```
for layer in base_model.layers:  
    layer.trainable = False  
  
print("Layers Frozen")
```

Explanation:

Feature extraction layers remain unchanged.

Only newly added layers will learn.

STEP 8 ADD CUSTOM CLASSIFIER

Purpose:

Create classifier for our dataset.

Code:

```
model = Sequential([
    base_model,
    Flatten(),
    Dense(
        128,
        activation='relu'
    ),
    Dropout(0.3),
    Dense(
        3,
        activation='softmax'
    )
])
```

Explanation:

Flatten

Converts features into vector.

Dense(128)

Learns dataset-specific patterns.

Softmax

Predicts class probabilities.

STEP 9 VIEW MODEL SUMMARY

Code:

```
model.summary()
```

Explanation:

Displays:

Layers

Parameters

Trainable Parameters

STEP 10 COMPILE MODEL

Purpose:

Configure training process.

Code:

```
model.compile(  
    optimizer='adam',  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)
```

STEP 11 TRAIN MODEL

Purpose:

Train custom classifier.

Code:

```
history = model.fit(  
    train_data,
```



```
        validation_data=val_data,  
        epochs=5  
    )
```

Explanation:

Only the custom Dense layers are trained.

VGG16 weights remain fixed.

STEP 12 EVALUATE MODEL

Purpose:

Check model performance.

Code:

```
loss, accuracy = model.evaluate(  
    val_data  
)  
  
print(  
    "Accuracy =",  
    accuracy  
)
```

STEP 13 PLOT ACCURACY CURVE

Code:

```
plt.plot(  
    history.history['accuracy']  
)  
  
plt.plot(  
    history.history['val_accuracy']  
)
```

```
plt.legend(  
    ['Training', 'Validation']  
)  
  
plt.title("Accuracy Curve")  
  
plt.show()
```

STEP 14 PLOT LOSS CURVE

Code:

```
plt.plot(  
    history.history['loss']  
)  
  
plt.plot(  
    history.history['val_loss']  
)  
  
plt.legend(  
    ['Training', 'Validation']  
)  
  
plt.title("Loss Curve")  
  
plt.show()
```

STEP 15 PREDICT NEW IMAGE

Purpose:

Classify unseen image.

Code:

```
from tensorflow.keras.preprocessing import image  
  
img = image.load_img(  

```

```
    "test.jpg",
    target_size=(224,224)
)

img_array = image.img_to_array(img)

img_array = img_array/255.0

img_array = np.expand_dims(
    img_array,
    axis=0
)

prediction = model.predict(
    img_array
)

print(prediction)
```

UNDERSTANDING TRANSFER LEARNING

Traditional CNN

↓

Train Everything From Scratch

↓

Needs Large Dataset

Transfer Learning

↓

Use Pretrained Model

↓

Keep Learned Features

↓

Train Only New Layers



Needs Less Data



Faster Training

KEY CONCEPTS LEARNED

- ✓ Transfer Learning
 - ✓ VGG16
 - ✓ ImageNet Weights
 - ✓ Feature Extraction
 - ✓ Freezing Layers
 - ✓ Dense Classifier
 - ✓ Loading .weights.h5
 - ✓ Dataset Reading
 - ✓ Training
 - ✓ Evaluation
 - ✓ Prediction
-

EXPECTED VIVA QUESTIONS

1. What is Transfer Learning?
2. Why use VGG16?
3. What is ImageNet?

4. Why are layers frozen?
5. What happens if layers are not frozen?
6. Difference between CNN and Transfer Learning?
7. Why is include_top=False used?
8. Why is VGG16 faster than training from scratch?
9. What is Feature Extraction?
10. What is Fine Tuning?